

Advanced Compiler Engineering: Project Report

Roshan Y Singh

1 Implemented Compiler Path

The requested compiler-side integration is present inside a real Triton source tree in this workspace:

- pass definition in `triton-src/include/triton/Dialect/TritonGPU/Transforms/Passes.td`
- pass declaration in `triton-src/include/triton/Dialect/TritonGPU/Transforms/Passes.h`
- pass implementation in `triton-src/lib/Dialect/TritonGPU/Transforms/F2SharedLayout.cpp`
- build integration in TritonGPU transforms CMake files
- runnable binaries in `triton-src/build-llvm/bin/triton-opt` and `triton-src/build-llvm/bin/triton-translate`

The pass is exposed as:

```
triton-opt --triton-gpu-f2-shared-layout
```

This is a real TTGIR pass, not a Python post-processor. The output is modified TTGIR printed by `triton-opt`, and that output continues to compile through Triton's downstream pipeline.

2 How The Pass Works

TableGen registration

The pass is registered exactly through Triton's normal MLIR-style pass machinery:

```
def TritonGPUF2SharedLayout : Pass<"triton-gpu-f2-shared-layout", "mlir::ModuleOp"> {  
  let summary = "Rewrite Triton shared-memory encodings using an F2 swizzle search";  
  let constructor = "mlir::createTritonGPUF2SharedLayoutPass()";  
  let options = [  
    Option<"maxSwizzleBits", "f2-max-swizzle-bits", "int32_t", "3", "...">,  
    Option<"emitRemarks", "f2-emit-remarks", "bool", "true", "...">  
  ];  
}
```

Rewrite logic

The implementation in `F2SharedLayout.cpp` works in four stages:

1. walk the module and collect every tensor type that uses `#triton_gpu.shared`
2. infer a representative logical 2-D tile shape for each distinct shared encoding

3. search Triton-representable F_2 swizzles by varying the shared-layout parameters that Triton actually prints and accepts: `perPhase` and `maxPhase`
4. rewrite all matching tensor types in function signatures, block arguments, and operation results, then re-verify the module

The key rewrite step is:

```
auto choice = findBestRepresentableF2(
    static_cast<int>(logicalShape[0]),
    static_cast<int>(logicalShape[1]),
    maxSwizzleBits);

auto newEncoding = triton::gpu::SharedEncodingAttr::get(
    ctx, originalEncoding.getVec(), choice.perPhase, choice.maxPhase,
    order, originalEncoding.getCTALayout(),
    originalEncoding.getHasLeadingOffset());
```

The search objective is intentionally simple and explicit. For each candidate swizzle, the pass:

- computes a bank-address rank proxy over F_2
- estimates row-wise and column-wise bank conflicts on the logical tile
- prefers higher rank first, then fewer simulated conflicts, then a larger representable swizzle depth

The pass preserves `vec`, `order`, CTA layout, and `hasLeadingOffset`; it only changes the swizzle-related shared-layout parameters. That keeps the rewritten IR within Triton’s own `#triton.gpu.shared` encoding space.

3 GEMM TTGIR Validation

The end-to-end validation flow for a standalone GEMM TTGIR module completed as follows:

1. input TTGIR: `ttgir_inputs/matmul_kernel.ttgir.mlir`
2. pass output TTGIR: `ttgir_outputs/matmul_kernel_f2_from_pass.ttgir.mlir`
3. PTX from original TTGIR: `ptx_outputs/matmul_kernel_orig.ptx`
4. PTX from rewritten TTGIR: `ptx_outputs/matmul_kernel_f2.ptx`

The important compiler-side checks are:

- `triton-opt --help` lists `--tritongpu-f2-shared-layout`
- running the pass on the GEMM TTGIR succeeds and emits rewritten TTGIR
- the rewritten TTGIR compiles onward with `triton-translate`
- the generated PTX is not identical to the original PTX

For example, the first shared-store address block changes from the original address construction `xor.b32 %r519, %r518, %r1` to the rewritten form `xor.b32 %r519, %r3, %r6`. This shows that the pass affects real shared-memory addressing in generated code, not only the textual TTGIR output.

Using the same simplified objective used during the search, the GEMM module’s static TTGIR-side proxy improves from **416** to **384** (**7.7%** lower).

4 Pass-Driven GEMM Hardware Profiling

Experiment design

To measure the pass itself rather than a separate prototype kernel family, a small CUDA Driver API launcher was added in `ptx_matmul_runner.cpp`. It loads either:

- `ptx_outputs/matmul_kernel_orig.ptx`, or
- `ptx_outputs/matmul_kernel_f2.ptx`

and launches the exact Triton-generated kernel `matmul_kernel_0d1d2d3de4de5de6de7c8de9c10de11c` for the same 128x256x32 GEMM tile that produced the original TTGIR example.

Nsight Compute was then run directly on this launcher with:

- `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`
- `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`
- `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`

Nsight Compute results

Table 1: Exact pass input vs pass output profiling for the GEMM PTX

Kernel	Total Conflicts	Load Conflicts	Store Conflicts	GPU Time Sum (ns)
Original TTGIR/PTX	1,052	16	1,036	168,896
Pass-output TTGIR/PTX	1,035	12	1,026	169,792

Relative to the original TTGIR/PTX, the pass-output kernel:

- reduces total bank conflicts by **1.62%**
- reduces load-side bank conflicts by **25.0%**
- reduces store-side bank conflicts by **0.97%**
- changes Nsight GPU time by only **+0.53%**

The key requested condition is satisfied here: for the actual pass-emitted output MLIR compiled onward to PTX, the shared-memory bank-conflict metric is better than for the input MLIR/PTX.

Direct timing sweep

To pair the hardware counter result with a non-Nsight timing check, the same PTX runner was executed directly for 5,000 serial kernel launches.

Table 2: Direct timing of original vs pass-output PTX

Kernel	Per-Iteration Time (ms)	Relative to Original
Original TTGIR/PTX	0.1528	baseline
Pass-output TTGIR/PTX	0.1512	1.04% faster

This direct timing result is small, but it points in the same favorable direction as the bank-conflict measurement for the standalone pass-driven GEMM case.

5 Whole-BERT Layout-Family Profiling

The workspace also contains a larger live BERT experiment (part of the final presentation) using the existing custom runner `bert_real_model_with_custom_layouts.py`. That path replaces `nn.Linear` layers with a custom CUDA GEMM and compares a naive layout to the same F₂ layout family. It is useful for system-level behavior, but it is not yet the same thing as running `triton-opt` on a saved whole-BERT TTGIR module and then executing the rewritten module.

The measured configuration was:

- `model = bert`
- `tilek = 64`
- `sequence length = 128`
- `batch size = 1`

Live Nsight Compute results

Table 3: Whole-BERT layout-family profiling

Layout	Total Conflicts	Load Conflicts	Store Conflicts	GPU Time Sum (ns)
Naive (layout 0)	14,132,340	524,246	13,608,094	222,987,520
F ₂ family (layout 2)	13,251,400	691,668	12,559,732	223,047,040

Relative to naive, the F₂ family layout:

- reduces total bank conflicts by **6.23%**
- reduces store-side bank conflicts by **7.70%**
- increases load-side bank conflicts by **31.94%**
- leaves summed kernel GPU time essentially unchanged

Direct whole-model timing

Table 4: Direct whole-BERT latency

Layout	Per-Iteration Time (ms)	Relative to Naive
Naive (layout 0)	195.80	baseline
F ₂ family (layout 2)	222.39	13.58% slower

So the larger BERT experiment still shows a real bank-conflict win for the F₂ family, but it does not show an end-to-end latency win on this machine.

6 Interpretation

The current evidence supports the following conclusions:

1. the requested Triton pass path is real and functional: `Passes.td`, `Passes.h`, `F2SharedLayout.cpp`, and `triton-opt` are all wired up

2. the pass emits modified TTGIR that remains valid and compiles onward to PTX
3. on the exact pass-driven GEMM experiment, the pass-output kernel is better than the input kernel on the shared-memory bank-conflict metric
4. on that same standalone GEMM experiment, the direct timing result is also slightly favorable to the pass output
5. on the larger BERT layout-family experiment, bank conflicts improve but whole-model latency does not

In short, the pass itself now has a direct hardware-backed before/after win on its output kernel, while the larger whole-model story remains more complicated.

7 Artifacts Produced

The main output artifacts from this round are:

- `ttgir_outputs/matmul_kernel_f2_from_pass.ttgir.mlir`
- `ptx_outputs/matmul_kernel_orig.ptx`
- `ptx_outputs/matmul_kernel_f2.ptx`
- `ptx_matmul_runner.cpp`
- `hardware_results/server_verification/ncu_matmul_pass_orig_raw.csv`
- `hardware_results/server_verification/ncu_matmul_pass_f2_raw.csv`
- `hardware_results/server_verification/matmul_pass_ncu_summary.json`
- `hardware_results/server_verification/matmul_pass_orig_timing_serial.txt`
- `hardware_results/server_verification/matmul_pass_f2_timing_serial.txt`
- `hardware_results/server_verification/real_bert_layout0_tk64_live.json`
- `hardware_results/server_verification/real_bert_layout2_tk64_live.json`
- `hardware_results/server_verification/ncu_real_bert_layout0_tk64_live_raw.csv`
- `hardware_results/server_verification/ncu_real_bert_layout2_tk64_live_raw.csv`
- `hardware_results/server_verification/real_bert_layout0_tk64_perf10.json`
- `hardware_results/server_verification/real_bert_layout2_tk64_perf10.json`

8 Current Boundary

One strict compiler-side stage is still open: taking a saved whole-BERT TTGIR module, running `triton-opt --tritongpu-f2-shared-layout` on that module directly, compiling the rewritten whole-model TTGIR onward, and then profiling that exact pass output end to end. The Triton pass itself is no longer the blocker. The missing piece is a saved whole-model BERT TTGIR artifact that can be replayed through the same pass pipeline as the standalone GEMM case.

What *is* completed is:

- the real Triton TableGen pass integration
- TTGIR rewrite and PTX generation for a standalone GEMM kernel
- exact input-vs-output hardware profiling for the pass-driven GEMM kernel
- whole-BERT layout-family profiling on the live GPU